

SF2524
Matrix computations for large-scale systems
Homework 3

Group 3
Ville Wassberg
Klara Zimmerman

December 2024

Problem 1

a)

In this problem, we implement the basic QR-method, where the idea is to compute the QR-factorization of a matrix $A = QR$, and then change the order of the multiplication of Q and R . The output after k iterates will then be matrices T and U , such that $A \approx UTU^*$ where T is upper triangular and $U^*U = I$, ie the Schur factorization of A . We measure the error by looking at the absolute value of the largest element of the sub-diagonal of T , which is ideally 0.

We run the Basic QR-method using different matrices A_α , which are $m \times m$ matrices shifted with parameter $\alpha \in (0, \infty)$. We let α vary between 0 and 10^5 , and measure the error at each iteration of the QR-method. This is plotted in Figure 1 along with the predicted number of iterations.

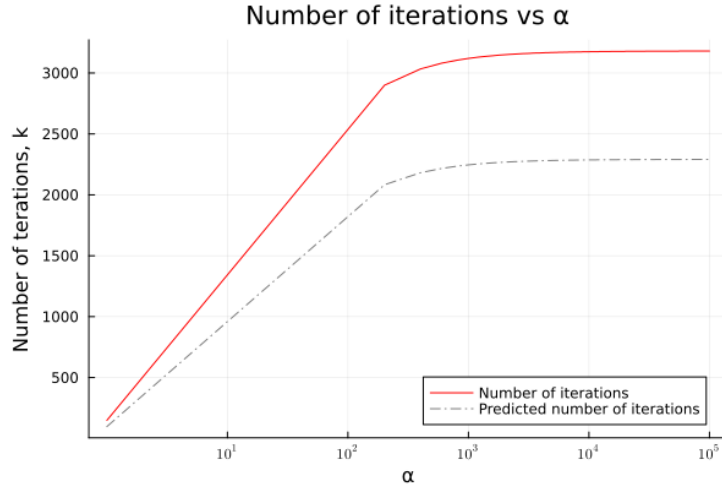


Figure 1: Number of iterations of the Basic QR-method needed until error of 10^{-10} is reached, both theoretical prediction and numerical results.

b)

The predicted number of iterations of the QR-method is found by examining the true eigenvalues of A_α . We know that the error of the sub-diagonal elements of T decreases proportionally to $|\beta|^n = |\lambda_i/\lambda_j|^n$ after n iterations, where $\lambda_i < \lambda_j$ are eigenvalues of A_α .

To find the upper bound of $|\beta|$, we thus look at the eigenvalue pair $(\lambda_i < \lambda_j)$ of A_α such that the ratio $|\lambda_i/\lambda_j|$ is maximized.

Consequently, the error will be dominated by the two closest eigenvalues. For large α , these turned out be the two largest eigenvalues λ_{m-1} and λ_m .

c)

The number of iterations needed for a particular accuracy (TOL) is calculated using the formula $k = \ln(TOL)/\ln(|\beta|)$, which is plotted together with our results in Figure 1.

As we can see in Figure 1, the number of iterations needed to achieve an error below the tolerance 10^{-10} stabilizes as α increases. This is because the eigenvalues of A_α converge for large α , resulting in a consistent value of $|\beta|$ across iterations. For matrices A_α , when α large, the largest and second largest eigenvalues yield $\beta = \lambda_{m-1}/\lambda_m = 0.989998$. Note that the theoretical prediction is a proportional bound, hence it can be scaled appropriately.

Problem 2

a)

In this problem, we look at Hessenberg reductions of matrices. We begin by generalizing the lemma about Householder reflectors from the lecture notes.

Lemma 1 (Generalized Lemma about Householder Reflectors). *Given a vector $x \in R^n$ and a vector $y \in R^n$ with $y \neq 0$ and $x \neq 0$, let $\rho = \pm 1$ and $\alpha = \rho||x||/||y||$. Let*

$$u = \frac{x - \alpha y}{||x - \alpha y||} = \frac{z}{||z||}.$$

Then the matrix $P = I - 2uu^T$ is a Householder reflector and $Px = \alpha y$.

Proof. We derive this lemma in a very similar way to the derivation of Lemma 3.2.3 in Block 3 of the lecture notes.

Let $u = \frac{x - \alpha y}{||x - \alpha y||} = \frac{z}{||z||}$. We know that since u is normalized, P is a Householder reflector.

We now want to find α such that $Px = \alpha y$ holds. The left hand side is

$$Px = (I - 2uu^T)x = x - 2uu^T x = x - 2 \frac{z}{||z||} \frac{z^T}{||z||} x = x - 2 \frac{z^T x}{z^T z} z. \quad (1)$$

If (1) is equal to $x - z$, then we get $Px = x - z = x - (x - \alpha y) = \alpha y$, and the theorem is proved. Thus, want to find z such that $2 \frac{z^T x}{z^T z} = 1$, or equivalently:

$$2z^T x = z^T z. \quad (2)$$

The left hand side of (2) is

$$2z^T x = 2(x - \alpha y)^T x = 2||x||^2 - 2\alpha x^T y,$$

and the right hand side of (2) is

$$z^T z = (x - \alpha y)^T (x - \alpha y) = ||x||^2 - 2\alpha x^T y + \alpha^2 ||y||^2.$$

Thus, (2) holds if and only if

$$\begin{aligned}
2\|x\|^2 - 2\alpha x^T y &= \|x\|^2 - 2\alpha x^T y + \alpha^2 \|y\|^2 \\
&\iff \\
\|x\|^2 &= \alpha^2 \|y\|^2 \\
&\iff \\
\alpha &= \pm \frac{\|x\|}{\|y\|}.
\end{aligned} \tag{3}$$

Hence, letting $\rho = \pm 1$, we get $\alpha = \rho \frac{\|x\|}{\|y\|}$, and the proof is complete. $\square$

We can note here that if $\|y\| = 1$, for example if $y = e_1$, then this result is the same as that of Lemma 3.2.3 in the lecture notes.

b)

See code.

c)

We now implement Algorithm 2 in the lecture notes ("efficient_hessenberg_red(A)" in our code) and compare the CPU-time to that of the naive (inefficient) algorithm in (b). The results are presented in Table 1. As we can see, Algorithm 2 is much more efficient, particularly as the dimension m of the matrix increases. For $m = 400$ Algorithm 2 is 5 times faster than the algorithm in (b).

m	CPU-time Algorithm 2	CPU-time of algorithm in (b)
10	0.00003	0.000058
100	0.00508	0.022223
200	0.05518	0.16444
300	0.21543	0.88498
400	0.39357	2.00594

Table 1: Comparison of CPU times in seconds for the efficient and inefficient Hessenberg reductions, for different matrix dimensions m .

d)

We now consider the matrix $A = \begin{pmatrix} 3 & 2 \\ \epsilon & 1 \end{pmatrix}$, and run one step of the shifted QR-method for different shifts σ , as we let $\epsilon \rightarrow 0$.

ε	$ \bar{h}_{2,1} $	
	$\sigma = 0$	$\sigma = a_{2,2}$
0.4	0.0961	0.0769
0.1	0.0311	0.0050
0.01	0.0033	4.4499e-5
0.001	0.0003	5.0e-7
10^{-4}	3.3e-5	5.0e-9
10^{-5}	3.3e-6	5.0e-11
10^{-6}	3.3e-7	5.0e-13
10^{-7}	3.3e-8	5.0e-15
10^{-8}	3.3e-9	5.0e-17
10^{-9}	3.3e-10	5.0e-19
10^{-10}	3.3e-11	5.0e-21
0	0	0

Table 2: Comparison of $|\bar{h}_{2,1}|$ for different ε values and σ settings.

When selecting $\epsilon = 0$ the shifted QR-method is reduced to the basic QR-method in this particular implementation, since no use of deflation is used here. Furthermore, when $\epsilon = 0$ the diagonal of A consists of the true eigenvalues, hence no steps are essentially needed. In the general case, selecting $\epsilon = 0$, the shifted QR-method becomes the basic QR-method combined with deflation.

The choice of shift σ is significantly better using $\sigma = a_{2,2} = 1$ than $\sigma = 0$. This shift corresponds to a true eigenvalue of the matrix A when $\epsilon = 0$ and we expect the method to converge in one step. Evidently, when ϵ is sufficiently small, this eigenvalue of A is still closer to $\sigma = 1$ than to $\sigma = 0$. This means that $\sigma = 1$ will be a better approximation, hence we expect $\bar{h}_{2,1}$ to converge to zero faster when using this shift.

Problem 3

a)

We now implement the Schur-Parlett algorithm for computing matrix functions using Schur-factorization (see code).

Letting $A = \begin{pmatrix} 1 & 4 & 4 \\ 3 & -1 & 3 \\ -1 & 4 & 4 \end{pmatrix}$, we can compute

$$f(A) = \sin(A) = \begin{pmatrix} 0.802329 & -0.270046 & -0.614218 \\ 0.120126 & 0.421095 & -0.396131 \\ -0.169106 & -0.0979604 & 0.535262 \end{pmatrix}.$$

b)

Let us now consider a random complex valued matrix $A \in \mathbf{C}^{100 \times 100}$. We compare the CPU-time of the Schur-Parlett algorithm for $f(A) = A^N$, with the naive algorithm that computes each matrix multiplication. These results are plotted in Figure 2. Clearly, for $N > 100$, the Naive version of the algorithm takes longer than the Schur-Parlett algorithm, and appears to increase linearly as N grows.

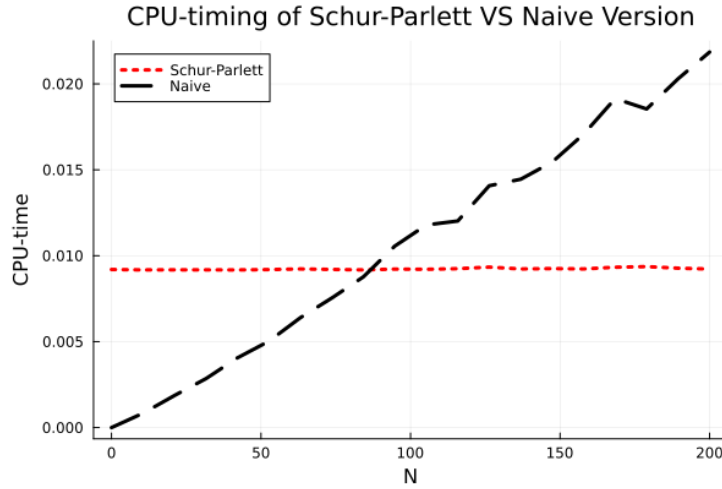


Figure 2: Comparison of CPU time (seconds) of the Schur-Parlett algorithm and the naive version of computing A^N , for increasing N .

c)

The computational complexity of the naive version of the exponential function of a matrix is determined by the dimension n of the matrix A , and the exponent

N . The matrix multiplication $A \times A$, where $A \in \mathbf{C}^{n \times n}$, has the computational complexity $\mathcal{O}(n^2)$. The naive algorithm computes such a multiplication N times, giving us the computational complexity $\mathcal{O}(n^2 N)$. Figure 2 shows the CPU-time of this algorithm for a constant n . As N increases, the CPU-time thus increases linearly, which is in line with our computational results.

As for the Schur-Parlett method, the theoretical computational complexity depends only on n . By analyzing the nested loops in the algorithm, we find that the complexity of the algorithm is expected to be $\mathcal{O}(n + (n-1) \cdot (n-p) \cdot (n-p) + n^3) = \mathcal{O}(n^3)$ corresponding to the three nested for loops and the matrix product of $Q F Q^*$. This explains the constant behaviour of the algorithm in terms of N , in our computations.

Problem 4

a)

To prove for the matrix $A = \begin{bmatrix} \pi & 1 \\ 0 & \pi + \varepsilon \end{bmatrix}$, $\varepsilon > 0$, that if p is a polynomial which interpolates with g in the eigenvalues of A , then $p(A) = g(A)$, we can begin by noticing that the eigenvalues of A are the diagonal elements of A and they are distinct. Now, suppose $p(\lambda_i) = g(\lambda_i)$, $i = 1, 2$, then by the Jordan Canonical Form definition

$$p(A) = X \begin{bmatrix} p(\pi) & 0 \\ 0 & p(\pi + \varepsilon) \end{bmatrix} X^{-1} = X \begin{bmatrix} g(\pi) & 0 \\ 0 & g(\pi + \varepsilon) \end{bmatrix} X^{-1} = g(A),$$

where X is a corresponding invertible matrix in the Jordan decomposition of A . In other words, since A is diagonalizable, the derivatives in the Jordan blocks are non-existent, and $p(A)$ is therefore equal to $g(A)$.

When $p(z) = \alpha + \beta z$, we can find the formulas for α and β in terms of g . Since $p(\lambda_i) = \alpha + \beta \lambda_i = g(\lambda_i)$, $i = 1, 2$, we have

$$\begin{cases} \alpha = g(\pi) - \beta \pi \\ \beta = \frac{g(\pi + \varepsilon) - \alpha}{\pi + \varepsilon} = \frac{g(\pi + \varepsilon) - g(\pi) + \beta \pi}{\pi + \varepsilon} \end{cases} \Rightarrow \begin{cases} \alpha = g(\pi) - \pi \frac{g(\pi + \varepsilon) - g(\pi)}{\varepsilon} \\ \beta = \frac{g(\pi + \varepsilon) - g(\pi)}{\varepsilon} \end{cases}.$$

b)

To find the exact formula for $\exp(A)$ using the above we can express X explicitly as a matrix with corresponding eigenvectors as columns. Thus X and X^{-1} can respectively be written as

$$X = \begin{bmatrix} 1 & 1 \\ 0 & \varepsilon \end{bmatrix}, \quad X^{-1} = \begin{bmatrix} 1 & -1/\varepsilon \\ 0 & 1/\varepsilon \end{bmatrix}.$$

This gives us an explicit formula for $\exp(A)$ which becomes

$$\exp(A) = X \begin{bmatrix} \exp \pi & 0 \\ 0 & \exp(\pi + \varepsilon) \end{bmatrix} X^{-1} = \begin{bmatrix} \exp(\pi) & \frac{\exp(\pi + \varepsilon) - \exp(\pi)}{\varepsilon} \\ 0 & \exp(\pi + \varepsilon) \end{bmatrix}, \quad \varepsilon > 0.$$

c)

Using this exact solution,

$$f(A) = \begin{bmatrix} \exp(\pi) & \frac{\exp(\pi + \varepsilon) - \exp(\pi)}{\varepsilon} \\ 0 & \exp(\pi + \varepsilon) \end{bmatrix},$$

we compare it to the solution obtained through the Jordan definition, as shown in the last example of section 4.1.2 of the course literature. Computing $\exp(A)$ using the Jordan definition involves extracting the diagonal matrix D containing the eigenvalues (λ_1, λ_2) of A , and the matrix V containing the corresponding

eigenvectors. Taking the exponential of the diagonal elements of D yields $\Lambda = \begin{bmatrix} \exp(\lambda_1) & 0 \\ 0 & \exp(\lambda_2) \end{bmatrix}$, and the matrix exponential is calculated by

$$F = V\Lambda V^{-1}.$$

The two solutions $f(A)$ and F are compared for different values of ϵ by taking the norm $\|f(A) - F\|$. This is plotted in Figure 3.

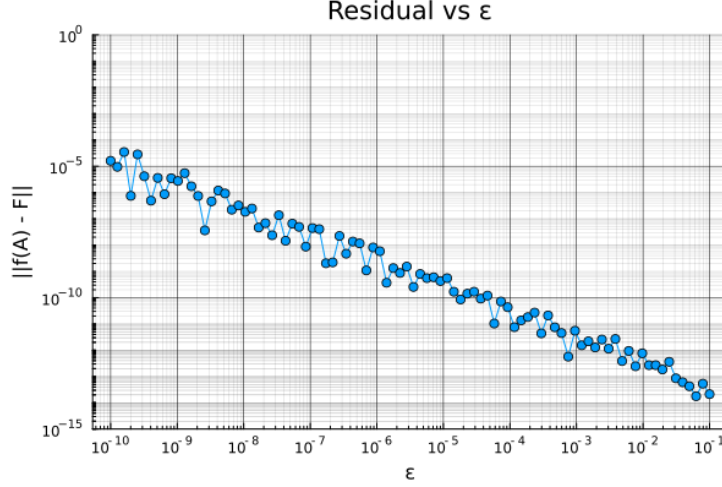


Figure 3: Relative error between the exact solution and computation following example 4.1.2.

For $\epsilon > 0$, A has distinct eigenvalues and is diagonalizable. Therefore, we assume that the eigenvalue matrix in the Jordan decomposition is diagonal. Had the eigenvalues been non-distinct, ie if ϵ had been equal to 0, the matrix containing the eigenvalues in the Jordan decomposition would need to be represented by a 2×2 Jordan block instead.

We can note that as $\epsilon \rightarrow 0$, the exact solution $f(A)$ becomes:

$$f(A) = \exp(A) = \begin{bmatrix} \exp(\pi) & \lim_{\epsilon \rightarrow 0} \frac{\exp(\pi + \epsilon) - \exp(\pi)}{\epsilon} \\ 0 & \exp(\pi + \epsilon) \end{bmatrix} = \begin{bmatrix} f(\pi) & f'(\pi) \\ 0 & f(\pi) \end{bmatrix}. \quad (4)$$

This is exactly the Jordan form of a matrix with non-distinct eigenvalues, confirming that this is the exact solution even as $\epsilon \rightarrow 0$.

However, in the case of F , we see can see that the eigenvectors of A become increasingly similar as $\epsilon \rightarrow 0$, since the eigenvalues approach each other. This

causes V to become more ill-conditioned, since the columns approach linear dependence. This increases the error when computing the inverse V^{-1} , and numerical errors in computing F . This explains the results in Figure 3, where we see a decreased error as ϵ increases.

Problem 5

In this problem we look at scaling and squaring with a Taylor-approximation. We consider the matrix $A \in R^{256 \times 256}$, and the Taylor approximation

$$T_m(A) = \sum_{k=0}^m \frac{1}{k!} A^k.$$

We let j be the number of matrix-matrix products in the squaring phase of scaling and squaring.

a)

In the scaling phase, we look at a Taylor approximation of order m , which requires squaring A $m - 1$ times. Adding the j matrix-matrix products in the squaring phase, we get a total of $j + m - 1$ matrix-matrix products. This is shown in Table 3.

	j=0	j=1	j=2	j=3	j=4	j=5	j=6	j=7
m=1	0	1	2	3	4	5	6	7
m=2	1	2	3	4	5	6	7	8
m=3	2	3	4	5	6	7	8	9
m=4	3	4	5	6	7	8	9	10
m=5	4	5	6	7	8	9	10	11
m=6	5	6	7	8	9	10	11	12
m=7	6	7	8	9	10	11	12	13

Table 3: The total number of matrix-matrix products in scaling and squaring.

Letting $P_{j,m}$ be the resulting approximation, we look at the logged error. This is shown in table 4.

	j=0	j=1	j=2	j=3	j=4	j=5	j=6	j=7
m=1	1.182	1.182	1.182	1.182	1.182	1.182	1.182	1.182
m=2	1.516	0.512	-0.134	-0.372	-0.659	-0.954	-1.253	-1.554
m=3	1.594	0.738	-0.321	-1.111	-1.788	-2.424	-3.042	-3.652
m=4	1.517	-0.134	-1.129	-2.150	-3.118	-4.053	-4.973	-5.884
m=5	1.328	-0.397	-1.924	-3.268	-4.540	-5.779	-7.000	-8.212
m=6	1.052	-1.045	-2.822	-4.468	-6.043	-7.583	-9.106	-10.620
m=7	0.703	-1.687	-3.785	-5.735	-7.613	-9.455	-11.279	-13.083

Table 4: $\log_{10} ||P_{j,m} - \exp(A)||$, where $\exp$ is the matrix exponential and $\log_{10}$ is the 10-base logarithm.

b)

	j=0	j=1	j=2	j=3	j=4	j=5	j=6	j=7
m=1	1.08e-5	1.12e-5	1.12e-5	1.11e-5	1.12e-5	1.13e-5	1.13e-5	1.0958e-5
m=2	6.66e-5	2.96e-4	5.11e-4	7.43e-4	0.0010	0.0013	0.0016	0.0018
m=3	3.56e-4	5.93e-4	9.11e-4	0.0011	0.0015	0.0018	0.0018	0.0023
m=4	6.40e-4	9.17e-4	0.0012	0.0015	0.0018	0.0021	0.0022	0.0026
m=5	9.48e-4	0.0013	0.0016	0.0018	0.0021	0.0024	0.0026	0.0030
m=6	0.0014	0.0017	0.0019	0.0022	0.0026	0.0027	0.0031	0.0034
m=7	0.0016	0.0021	0.0023	0.0026	0.0029	0.0032	0.0037	0.0036

Table 5: CPU-time of Scaling and Squaring using Taylor expansions.

c)

Based on Table 3 and Table 4, we would need 13 matrix-matrix multiplications to reach an accuracy of 10^{-12} . This is found using $m = 7$ and $j = 7$, and takes 0.0036 seconds to compute according to Table 5.

The first column of Table 5, where $j = 0$, corresponds to just doing a Taylor approximation. If we were to continue with larger values of m , we would find that $m = 25$ is needed to obtain an error smaller than 10^{-12} . This would require 24 matrix-matrix multiplications, and take approximately 0.0067 seconds. This is almost twice as many matrix-matrix multiplications and twice as much CPU-time as it takes to obtain the same accuracy with $m = 7$ if we let $j = 7$.

Problem 6

(a) 312: Householder reflectors that map x to the direction of e_1

A Householder reflector $P := I - uu^T$ where $u \in C^n$ and $\|u\| = 1$ is an $n \times n$ matrix such that Px "mirrors" a vector x over the plane that is perpendicular to u . In this video, we learn that P satisfies the equation

$$Px = \alpha e_1,$$

for $x \in R^n \setminus \{0\}$ if the following holds:

$$\begin{aligned}\rho &= \pm 1, \\ \alpha &= \rho \|x\|, \\ z &= x - \alpha e_1, \\ u &= \frac{z}{\|z\|}.\end{aligned}$$

This means that P reflects x in the direction of e_1 .

(b) 325 QR-factorization of Hessenberg via Givens derivation

In this video we are presented with a theorem showing that we can find a QR-factorization $A = QR$, of an unreduced Hessenberg matrix A , where $Q = G_1 \dots G_{n-1}$ and $R = H_n$. This is done by letting $H_1 = A$, and then iterating $H_{i+1} = G_i^T H_i$ for $i = 1, \dots, n-1$ where $G_i = G(i, i+1, c_i, s_i)$ is a Givens rotation matrix. The input to G is given by

$$\begin{aligned}c_i &= \frac{(H_i)_{i+1,i}}{r_i}, \\ s_i &= \frac{(H_i)_{i,i}}{r_i}, \\ r_i &= \sqrt{(H_i)_{i+1,i}^2 + (H_i)_{i,i}^2}.\end{aligned}$$

The proof of this theorem consists in showing that the matrix H_i is a reduced Hessenberg matrix where the upper left $(i-1) \times (i-1)$ block is upper triangular, resulting in a completely upper triangular matrix when $i = n$. Inserting all matrices into the recursion formula then proves that $A = G_1 \dots G_{n-1} H_n$ is a QR-factorization.

(c) 333: Shifted QR-method with exact shifts

One way of improving the convergence speed of the QR-method for a matrix A is through a shift. Letting $QR = H - \mu I$, where μ is an eigenvalue of A , we can prove that the QR method converges in just one iteration. Of course we do not

know the eigenvalues of A analytically, however if we approximate $\tilde{\mu} \approx \mu$, we will get

$$H_{k+1} = \begin{pmatrix} x & \dots & \dots & \dots & \dots & x \\ x & \ddots & & & & \vdots \\ & \ddots & \ddots & & & \vdots \\ & & \ddots & \ddots & & \vdots \\ & & & \ddots & \ddots & \vdots \\ & & & & x & x & x \\ & & & & \epsilon & \tilde{\mu} \end{pmatrix}$$

where ϵ is small. If the eigenvalue approximation is good, this will converge quickly.

(d) 344 Convergence of QR-method. Part 1. Simultaneous iteration: Normalized unnormalized

For the unnormalized simultaneous iteration (USI), the iteration step is $V^{k+1} = AV^k$, which on closed form becomes

$$V^{(k)} = A^{(k)}V^{(0)}. \quad (5)$$

For the normalized simultaneous iteration (NSI), the two iteration lines are:

$$\begin{aligned} w &= AQ^{(k)} \\ Q^{(k+1)}R^{(k+1)} &= w, \end{aligned} \quad (6)$$

which can be rearranged to

$$Q^{(k-1)} = AQ^{(k-2)}(R^{(k-1)})^{-1}. \quad (7)$$

By defining $Q^{(k)}R^{(k)} = V^{(k)}$, and plugging in (7), we derive

$$Q^{(k)}R := Q^{(k)}R^{(k)} \dots R^{(1)} = A^k Q^{(0)}.$$

This holds as long as the matrices $R^{(1)}, \dots, R^{(k)}$ are non-singular, and the QR-factorization is selected uniquely. Thus under these conditions, the iterations of the two methods are equivalent.

(e) 345 Convergence of QR-method. Part 2. Convergence unnormalized simultaneous iteration for special case

Using the unnormalized simultaneous iteration (USI) defined in (5), we now set up a special case where $A = \Lambda = \begin{pmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_n \end{pmatrix}$, is a diagonal matrix consisting of the eigenvalues, and $V^{(0)} \in \mathbb{R}^{n \times n}$ is lower triangular with 1:s on the diagonal.

By these assumptions, and the proposition that for the QR-factorization $QR = A$, the following holds:

- $q_1 r_{11} = a_1$,
- $q_2 r_{22} = (I - q_1 q_1^T) a_2$,
- $q_3 r_{33} = (I - q_1 q_1^T)(I - q_2 q_2^T) a_3$,

we can show that $Q = [q_1^{(k)}, \dots, q_n^{(k)}]$ converges towards $[e_1, \dots, e_n]$, ie the eigenvectors of A .

First we show that $q_1^{(k)}$ converges toward e_1 with a speed of $O(\frac{\lambda_2}{\lambda_1})^k$, which is then used to show that $q_2^{(k)}$ approaches e_2 with a speed $O(\frac{\lambda_2}{\lambda_1})^k + O(\frac{\lambda_3}{\lambda_2})^k$. Continuing in this manner, we see that $Q \rightarrow I$, where I is the eigenvector matrix of A .

(f) 423 Newton Square Root Method

To compute the principal square root of a matrix A we assume that A has no eigenvalues on the non-positive real line to ensure the existence of a well defined principal square root, and by using using Newton-Raphson's, NR, method for the scalar function $g(x) = x^2 - a$ to find the roots of $g(x)$, hence, finding $\sqrt{a}$ serves as a justification for the matrix version. By the NR we have $x_{k+1} = x_k - \frac{g(x_k)}{g'(x_k)} = \frac{1}{2}(x_k + ax_k^{-1})$, which is applied to A as: $X_{k+1} = \frac{1}{2}(X_k + AX_k^{-1})$, $X_0 = A$. Assuming A is symmetric, and hence diagonalisable, $A = V \Lambda V^{-1}$, $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ as well as defining the iterates $\Lambda_k = V^{-1} X_k V$, $k = 0, \dots$. Starting with $\Lambda_0 = V^{-1} X_0 V = V^{-1} A V = \Lambda$, gives by induction, $\Lambda_k = \frac{1}{2}(\Lambda_k + \Lambda \Lambda_k^{-1})$, which is a simultaneous diagonalization of the X_k matrices. Since the second derivative in the scalar case is 2 we use the fact that the method has quadratic local and almost global convergence to $\sqrt{a}$, and since the diagonal elements here are the update formulas for the scalar newton case of $g(x) = x^2 - a$ the convergence is inherited and thus, $\sqrt{A}$.

(g) 434 Full orthogonalization method, FOM.

FOM can be considered as a variant of the GMRES, and is derived using the normal equations. In particular, in GMRES we have the least squares problem, since

$$\min_{x \in \mathcal{X}_m(A, b)} \|Ax - b\| = \min_{z \in \mathbb{R}^m} \|AQ_m z - b\|,$$

which implies that the normal equations are satisfied

$$(A Q_m)^T \underbrace{(A Q_m z - b)}_{r_m} = 0.$$

This means the residual r_m is orthogonal to the Krylov subspace

$$r_m \perp \mathcal{K}_m(A, b),$$

hence, the residual is orthogonal to the Q_m matrix whose columns span the Krylov subspace. Then, by using the Arnoldi factorisation this can be derived into the linear system

$$z = H_m^{-1} e_1 \|b\|,$$

where H_m is the square Hessenberg matrix from the Arnoldi factorization and, hence, in general can be considered to have an inverse. This is the full orthogonalisation method.